

String Similarity Measures for Myanmar Language and Code Validation and Modification Problems in Java and Javascript for Programming Education

Khaing Hsu Wai

JUNE 7, 2025

Overview

In this talk, I will present **two** research projects:

01 String Similarity Measures for Myanmar Language
(master's)

**02 Code Validation and Modification Problems in
Java and Javascript for Programming Education**
(PhD)

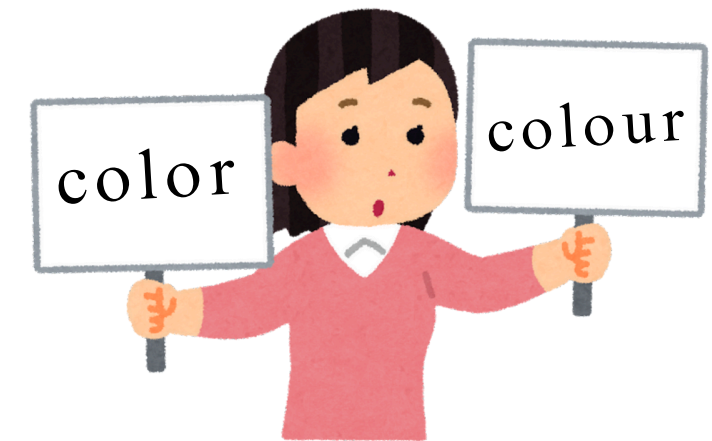
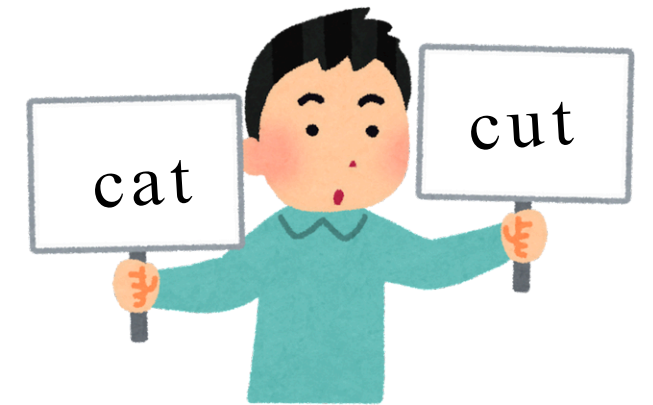
01 String Similarity Measures for Myanmar Language

(master's)

Papers: NSURL
JIIST

▼ Background

- String similarity measures **how similar** the two strings are.
- In language processing, it helps with searching words, correcting typos, and understanding similar meanings.
- In general, there are **three** main types of metrics to measure string similarity,
 - Edit-distance based metrics
 - Token-based metrics
 - Sequence-based metrics



▼ Contribution

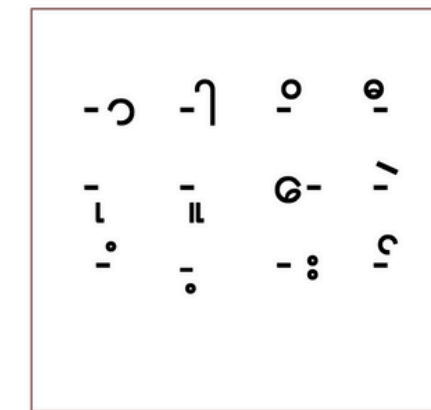
- Myanmar Language
 - is a tonal language
 - not only consonant can form a word
 - “က” - “dance” in English
 - but also one syllable can form a word
 - “ကြိုက်” - “like” in English
 - many phonetically similar sounds of characters and words
- As existing string similarity methods were designed for English or non-tonal languages, so they **fail** to handle Myanmar language properly.

Propose

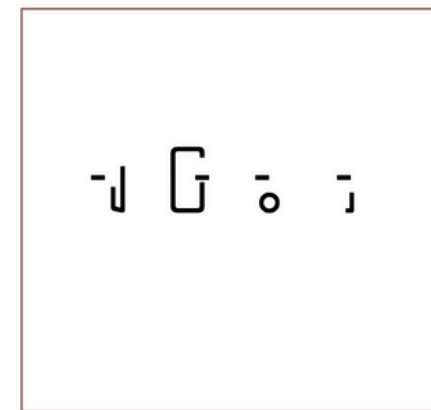
three mapping approaches with the existing **string similarity metrics** for Myanmar Language



Basic Consonant



Vowels



Medials

ကျောင်း

Example with combination of consonants, vowels and medials

▼ **Proposed Mapping**

- Phonetic Mapping
- Sound Mapping
- Vowel Position Mapping

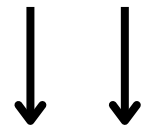
▼ **Six similarity measures**

- Edit-distance based metrics
 - Levenshtein Distance
 - Damerau-Levenshtein Distance
 - Hamming Distance
- Token-based metrics
 - Jaccard Similarity
 - Cosine Similarity
- Sequence-based Methods
 - JaroWinkler Similarity

▼ Levenshtein Distance

- Given two character strings s1 and s2, the edit distance between them is the minimum number of edit operations required to transform s1 into s2.
- The edit operations are insertion, deletion and substitution.

◦ 6mλ: - 6 m λ :



◦ m6λ: - m 6 λ :

Levenshtein Distance = 2

▼ Damerau Levenshtein Distance

- The Damerau-Levenshtein distance differs from the Levenshtein distance by including transpositions among its allowable operations in addition to three classical single character edit operations.

◦ 6mλ: - 6 m λ :



◦ m6λ: - m 6 λ :

Damerau Levenshtein Distance = 1

▼ Hamming Distance

- The Hamming distance between two strings of equal length measures the number of positions with mismatching characters.

◦ 3260 - 32 6○ 0



◦ 3263 - 32 6○ 3

Hamming Distance = 1

▼ Jaccard Similarity

- Jaccard similarity measures similarities between sets.

◦ s1 = 3260 → set A = {32, 0, 6○}

◦ s2 = 3263 → set B = {32, 3, 6○}

▪ $A \cap B = \{32, 6○\} \rightarrow 2$

▪ $A \cup B = \{32, 0, 3, 6○\} \rightarrow 4$

$$\text{Jaccard Similarity} = \frac{|A \cap B|}{|A \cup B|}$$

$$\text{Jaccard Similarity} = 2/4 = 0.5$$

▼ Cosine Similarity

- Cosine similarity measures how similar two texts are by looking at the angle between their word or character vectors.
- The value ranges from 0 to 1. (0=completely different, 1=exactly the same)
 - “3060” vs “30600” → Very similar → Cosine ≈ 0.67

$$\text{Cosine Similarity} = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\| \times \|\vec{B}\|}$$

▼ Jaro Winkler Similarity

- Jaro Winkler distance is a string metric measuring an edit distance between two sequences.
- The value ranges from 0 to 1. (0=completely different, 1=exactly the same)
 - 3060 - 30 60 0
 - 30600 - 30 60 00
 - $m = 2$ (matching characters)
 - $t = 0$ (transpositions)
 - $|s1| = |s2| = 3$

$$\text{Jaro} = \frac{1}{3} \left(\frac{2}{3} + \frac{2}{3} + \frac{2-0}{2} \right) = \frac{1}{3} (0.666 + 0.666 + 1) = \frac{2.332}{3} \approx 0.777$$

$$\text{Jaro}(s1, s2) = \frac{1}{3} \left(\frac{m}{|s1|} + \frac{m}{|s2|} + \frac{m-t}{m} \right)$$

- m = number of matching characters
- t = number of transpositions
- $|s1|, |s2|$ = lengths of the two strings

▼ Soundex

- Soundex algorithm is a phonetic algorithm.
- It is based on how close two words are depending on pronunciation.
- It converts words into codes based on how they sound, not how they are spelled.

Flower = F460

Flour = F460

Money = M500

Letters	Code
A,E,H,I,O,U,W,Y	0
B,F,P,V	1
C,G,K,Q,S,X,Z	2
L	4
M,N	5
R	6

Based on the idea of Soundex algorithm, there are **three proposed mappings for Myanmar Language.**

▼ Proposed Mapping - Phonetic Mapping

Letters	Code	Letters	Code
က ခ	က	ဝံ့	(delete)
ဂ ဃ	ဂ	လဲ ဤ ဧ ဝိ ဝီ ညံ	i
စ ဆ	စ	ကံ ပံ တံ	d
ဇ ဈ	ဇ	နံ မံ ဝံ	n
ဋ တ	တ	ဲ ရံ	e
ဌ ထ	ထ	ဥ ဦ ိ ူ	u
ဍ ဎ	ဍ	တ ဝါ	r
ဏ န	န	ဧ ဝေ	a
ဒ ဓ	ဒ	ံ ဝး	(delete)
ပ ဖ	ပ	ံ	(delete)
ဗ ဘ	ဘ	သြ ငြော် သြ သြော်	o
ယ ရ	ရ	ငင်း င	င
လ ဌ	လ	ါ ူ	s
သ သ	သ	ံ င် င ညံ	in
ျ ြ	y	? ! . * - = \ # " < > { } [] , + -	s

Table : Phonetic Mapping

- Same pronunciation words are grouped together.
- For example, ကလေး and ခလေး have same pronunciation.
- Some diacritics such as ဝံ (WaHswe) and ိ (Ha Hto), tone marks such as ဝံ (Aukmyit), ဝံ (Myanmar sign Virama) are considered to be removed.
- Source String = ပံစီး
- Target String = ပံစည်း
- Encoded Source String using Phonetic Mapping = ပံစီး
- Encoded Target String using Phonetic Mapping = ပံစည်း

▼ Proposed Mapping - Phonetic Mapping

Source String Target String	ပပ်စီး ပစ္စည်း	ပထဝီ ပထဝီ
Levenshtein Distance	3	0
Damerau-Levenshtein Distance	3	0
Hamming Distance	4	0
Jaro Winkler Distance	0.7968	1
Cosine Similarity	0.4999	1
Jaccard Similarity	0.4444	0.6

Table : Calculation of string similarity between two strings with Phonetic Mapping

▼ Proposed Mapping - Sound Mapping

Letters	Code	Letters	Code
က ခ ဂ ဃ င ဟ အ	က	ဝံ့	(delete)
ည	ည	ဣ ဤ ဧ ူ ည်	i
စ ဆ ဇ ဈ	စ	ကံ ပံ တံ	d
ဋ ဌ ဍ ဎ ဏ ပ တ ထ ဒ ဓ န	တ	နံ မံ	n
ပ ဖ ဗ ဘ မ	ပ	ဲရ်	e
ယ ရ	ရ	ဥ ဦ ဲ ဳ	u
လ ဌ	လ	တဝါ	r
သ သာ	သ	ဧ ူ	a
ျ ြ	y	့း	(delete)
ါ ဩ	s	ံ	(delete)
ှ ျှ	ှ	ဩ ဩ ဩ ဩ	o
ံ င် င ဉ်	in	? ! . * - = \ # " < > { } [] , + -	s

Table : Sound Mapping

- Sound Mapping is similar to Phonetic Mapping.
- The main different part of Phonetic mapping is at the Myanmar consonant.
- As the name of sound mapping, consonants, which have the same movements from mouth, lips and tongue, are grouped.
- For example, က ခ ဂ ဃ င ဟ အ (Ka Kha Ga Gha Nga Ha A) are clustered of က (Ka) group. ပ ဖ ဗ ဘ မ (Pa Pha Ba Bha Ma) are clustered of ပ (Pa) group etc.

▼ Proposed Mapping - Sound Mapping

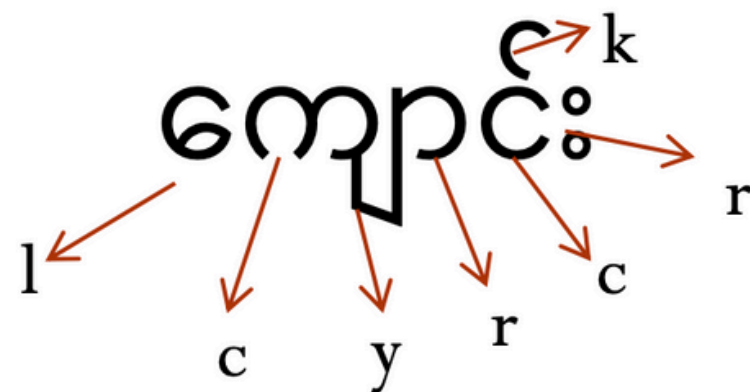
Source String Target String	ဖုရား ဘုရား	ပုရ ပုရ
Levenshtein Distance	1	0
Damerau-Levenshtein Distance	1	0
Hamming Distance	1	0
Jaro Winkler Distance	0.8667	1
Cosine Similarity	0.4999	1
Jaccard Similarity	0.6667	1

Table : Calculation of string similarity between two strings with Sound Mapping

▼ Proposed Mapping - Vowel Position Mapping

Letters	Code	Letters	Code
a-z A-Z	F	က-ခ	c
ချ	y	၎	p
ေ	l	က ဝါ ဝု ဝး	r
ိ ဝိ ဝဲ ဝံ	u	ဝ ဝ ဝ ဝ	d
်	k	၁၁၁	s
က ဘျ ဂ ညီ န သြ ငြော် သ ဉ် ဖျ ဇ်	i	? ! . * - = \ # " < > { } [] , + -	\$
ဝ-ဇ	n	0-9	D

Table : Vowel Position Mapping



- The third mapping based on the syllable formation of Myanmar language.
- The vowels written on the left side of the consonant are under the left (l) group, the right side vowels are under the (r) group, the upper vowels are under the (u) group, the lower vowels are under the (d) group respectively.

▼ Proposed Mapping - Vowel Position Mapping

Source String Target String	ကျောင်း ချောင်း	cylrckr cylrckr
Levenshtein Distance	1	0
Damerau-Levenshtein Distance	1	0
Hamming Distance	1	0
Jaro Winkler Distance	0.9047	1
Cosine Similarity	0.4999	1
Jaccard Similarity	0.75	1

Table : Calculation of string similarity between two strings with Vowel Position Mapping

02

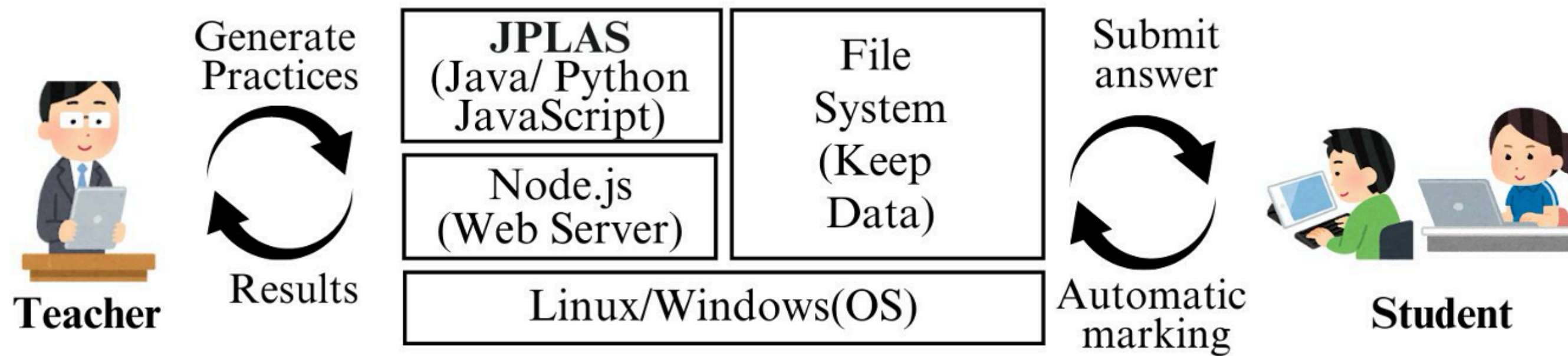
Code Validation and Modification Problems in Java and Javascript for Programming Education

(PhD)

Thesis

▼ Programming Learning Assistant System (PLAS)

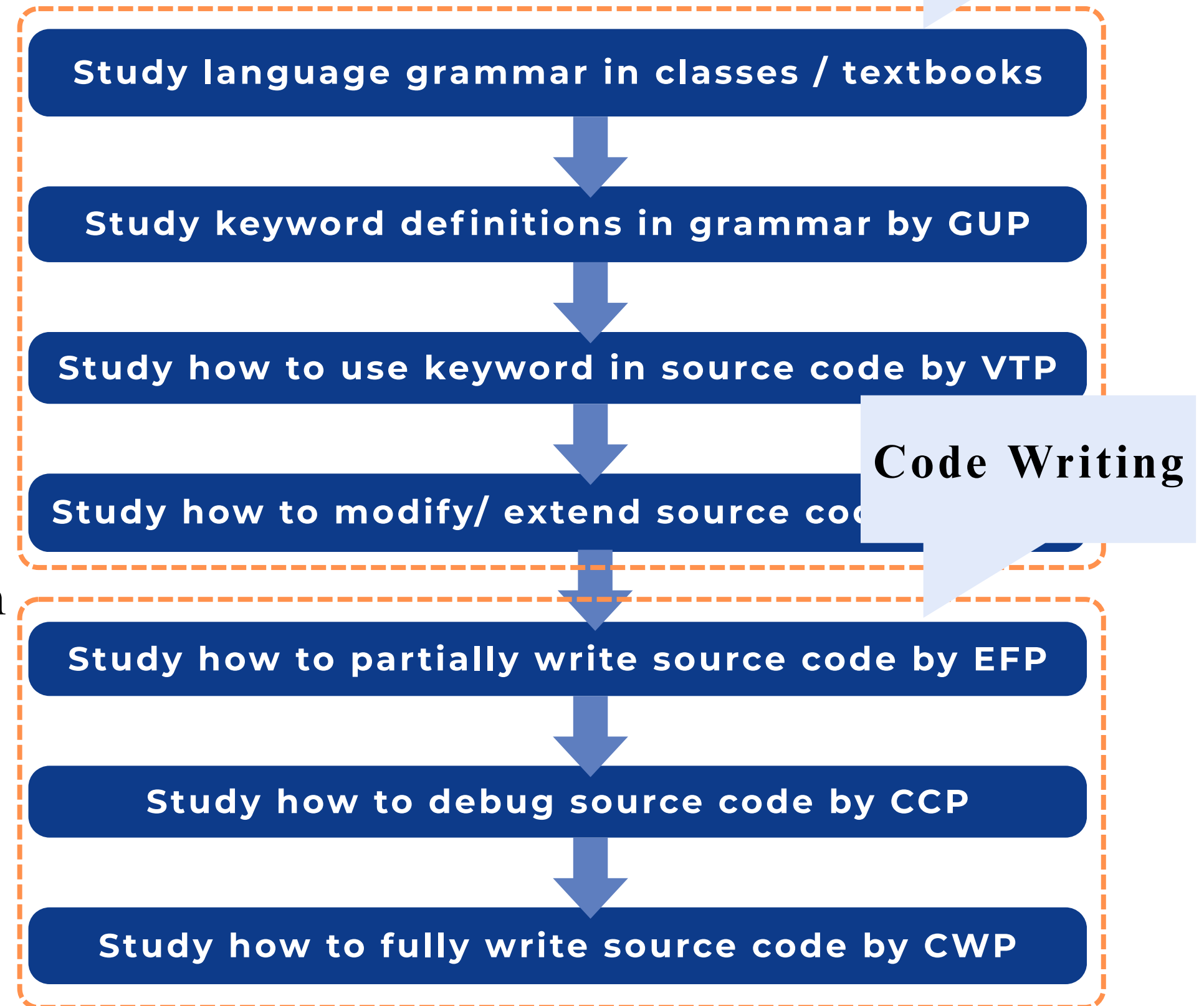
- PLAS is a web-application system to assist self-studies of Java programming for code reading and code writing skills by novice students.



▼ Step-by-Step Programming Study

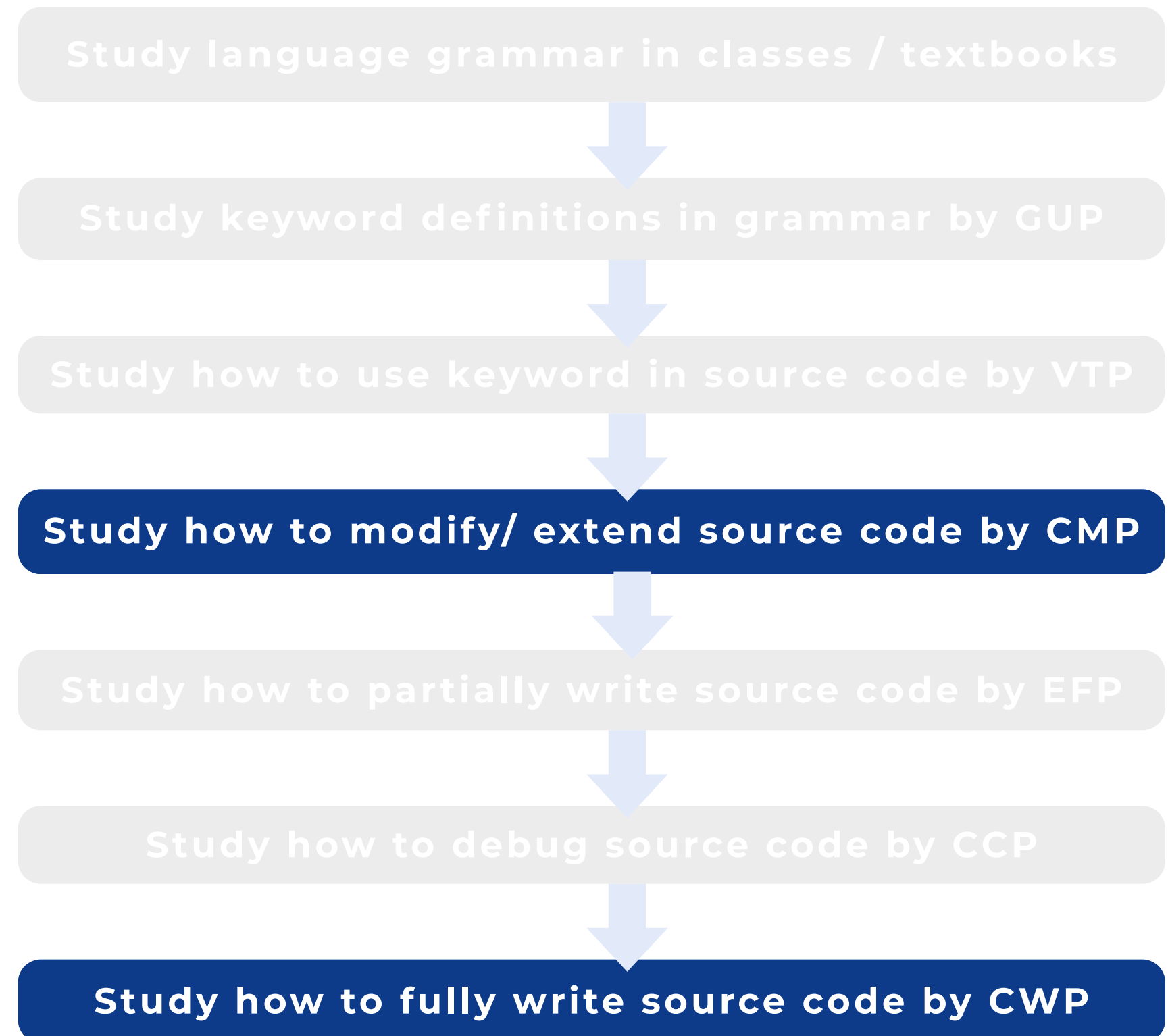
Code Reading

- PLAS offers different types of exercise problems for **step-by-step programming** study.
 - GUP : Grammar-Concept Understanding Problem
 - VTP : Value Trace Problem
 - CMP : Code Modification Problem
 - EFP : Element Fill-in-blank Problem
 - CCP : Code Completion Problem
 - CWP : Code Writing Problem



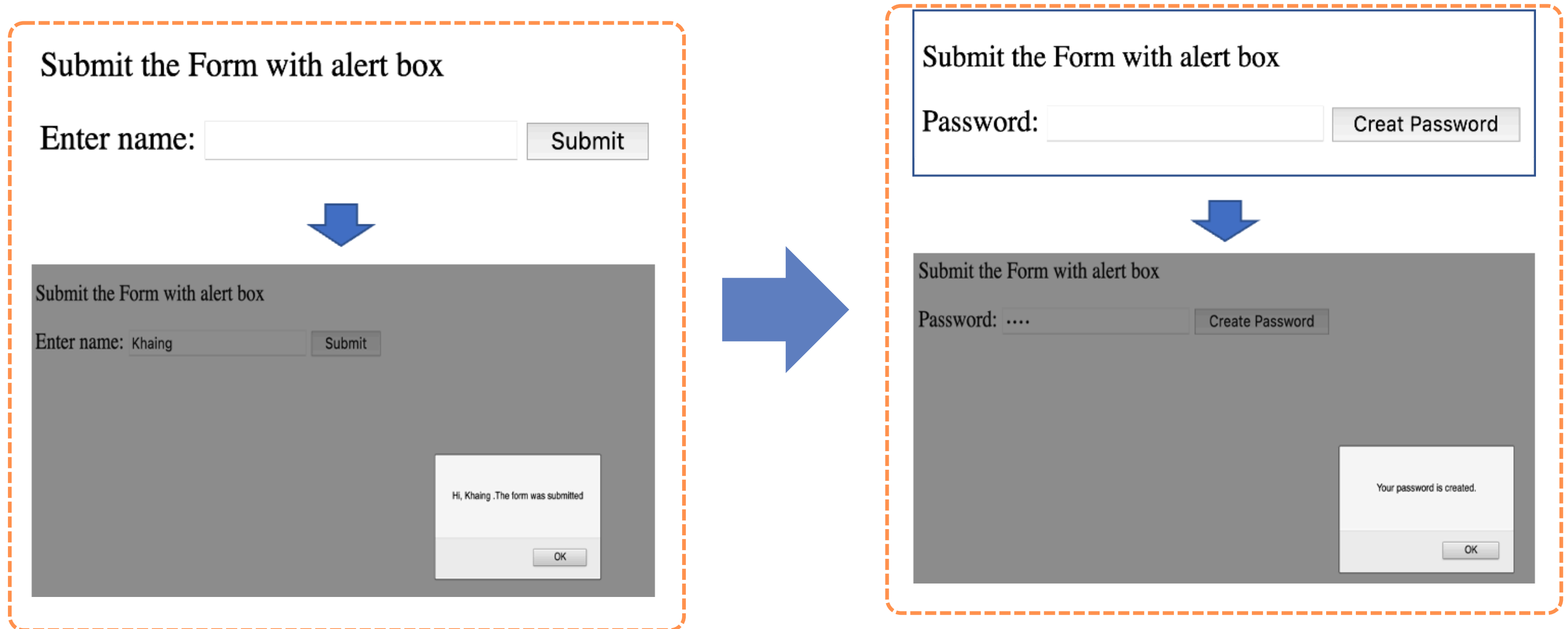
▼ Step-by-Step Programming Study

- PLAS offers different types of exercise problems for step-by-step programming study.
 - GUP : Grammar-Concept Understanding Problem
 - VTP : Value Trace Problem
 - **CMP : Code Modification Problem**
 - EFP : Element Fill-in-blank Problem
 - CCP : Code Completion Problem
 - **CWP : Code Writing Problem**



▼ Proposal of Code Modification Problem(CMP)

- In the CMP instance, a set of [web page screenshots](#) and a [source code](#) are shown to a student.



▼ Source Code

Submit the Form with alert box

Password:

Creat Password



Submit the Form with alert box

Password:

Create Password

Your password is created.

OK

```
01:<html>
02:<body>
03:<p>Submit the Form with alert box</p>
04:<form name="myForm" onsubmit="myFunction()">
05:    Enter name: <input type="text" name="fname">
06:        <input type="submit" value="Submit">
07:</form>
08:<script>
09:function myFunction() {
10:    var x = document.forms["myForm"]["fname"].value;
11:    alert("Hi, "+x+" .The form was submitted");
12:}
13:</script>
14:</body>
15:</html>
```

▼ Output Form

```
01 <html>
02 <body>
03 <p>Submit the Form with alert box</p>
04 <form name="myForm" onsubmit="myFunction()">
05   Enter name: <input type="text" name="fname">
06   <input type="submit" value="Submit">
07 </form>
08 <script>
09 function myFunction() {
10   var x = document.forms["myForm"]["passwrld"].value;
11   alert("Hi, "+x+" .The form was submitted");
12 }
13 </script>
14 </body>
15 </html>
```

```
01 <html>
02 <body>
03 <p>Submit the Form with alert box</p>
04 <form name="myForm" onsubmit="myFunction()">
05   Password: <input type="password" name="passwrld">
06   <input type="submit" value="Create Password">
07 </form>
08 <script>
09 function myFunction() {
10   var x = document.forms["myForm"]["passwrld"].value;
11   alert("Your password is created.");
12 }
13 </script>
14 </body>
15 </html>
```

Answer

Answer

File Save



➤ **Incorrect answer**



➤ **Correct answer**

▼ **Answer Code Validation for Code Writing Problem (CWP)**

- Implementation of Answer Code Validation Program for Code Writing Problem (CWP)
- Intermediate State Testing for Fundamental Algorithm Assignments
- Implementation of Dynamic Test Data Generation Algorithm for CWP
- Implementation of Naming Rules Checking Function in Code Validation Program

▼ Code Writing Problem (CWP)

- The student needs to **write** a whole **source code** from scratch that **satisfies** the **specifications** given by a **test code**.
- The **correctness** of the source code is checked by running the test code on **JUnit**.
- JUnit is a **unit testing** open source framework for Java programming language, which is used for writing and running tests.

JUnit-based Test Code

```
1: package p1;
2: import static org.junit.Assert.*;
3: import org.junit.Test;
4: public class MyMathTest {
5:     @Test
6:     public void testPlus(){
7:         MyMath ma = new MyMath();
8:         int codeInput1 = 1;
9:         int codeInput2 = 4;
10:        int codeOutput = ma.plus (codeInput1, codeInput2);
11:        int expOutput = 5;
12:        try {
13:            assertEquals("1:Two input case:", expOutput, codeOutput);
14:        } catch (AssertionError ae) {
15:            System.out.println(ae);
16:        }
17:    }
18: }
```

Package and Junit Library

Test class declaration

Signal to Junit that it is a test case

Test case/ Test method

▼ Test Code and Source Code

Source Code

```
1: package p1;
2: public class MyMath{
3:     public int plus (int a, int b){
4:         return (a+b);
5:     }
6: }
```

Junit-based Test Code

```
1: package p1;
2: import static org.junit.Assert.*;
3: import org.junit.Test;
4: public class MyMathTest {
5:     @Test
6:     public void testPlus(){
7:         MyMath ma = new MyMath();
8:         int codeInput1 = 1;
9:         int codeInput2 = 4;
10:        int codeOutput = ma.plus(codeInput1, codeInput2);
11:        int expOutput = 5;
12:        try {
13:            assertEquals("1:Two input case:", expOutput, codeOutput);
14:        } catch (AssertionError ae) {
15:            System.out.println(ae);
16:        }
17:    }
18: }
```

returning data type

class name

data type and number of arguments

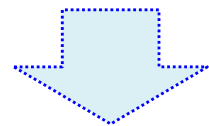
Assertion Method: to determine Pass or Fail

Expected Value

Actual Value

▼ Useful of Test Code

- Test code gives necessary information to implement the source code:
 - names of classes, access modifiers of classes
 - methods, variables
 - data types of variables, arguments, returning data of methods
 - exception handling



Test code can help students to write highly reliable and bug-free source code.

CWP Basic

Theme: neat

Submit

Next

Source Code

```
1 package p1;
2 import static org.junit.Assert.assertEquals;
3 import java.io.ByteArrayOutputStream;
4 import java.io.PrintStream;
5 import org.junit.Test;
6
7 import report.Report;
8
9 public class HelloWorld{
10     public static void main(String[] outs){
11         System.out.println("Hello World!");
12     }
13 }
```

1 The students can write the source code in designated place.

Test Code

```
1 package p1;
2 import static org.junit.Assert.assertEquals;
3 import java.io.ByteArrayOutputStream;
4 import java.io.PrintStream;
5 import org.junit.Test;
6
7
8
9 public class HelloWorldTest {
10
11     @Test
12     public void testMain() {
13         ByteArrayOutputStream bos = new ByteArrayOutputStream() ;
14         PrintStream bout = new PrintStream(bos);
15         System.setOut(bout);
16         HelloWorld.main(null);
17         String[] outs = bos.toString().split(System.lineSeparator());
18         //System.out.println(outs.length);
19         assertEquals("Hello World!", outs[outs.length-1]);
20     }
21
22 }
23
24
25
```

2 Submit the source code to validate

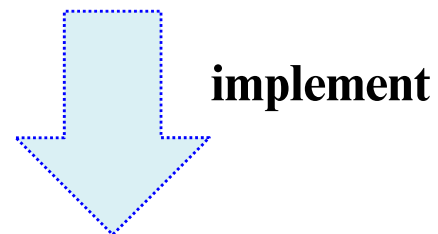
JUnit version 4.3
OK (1 test)

3 See the validation results

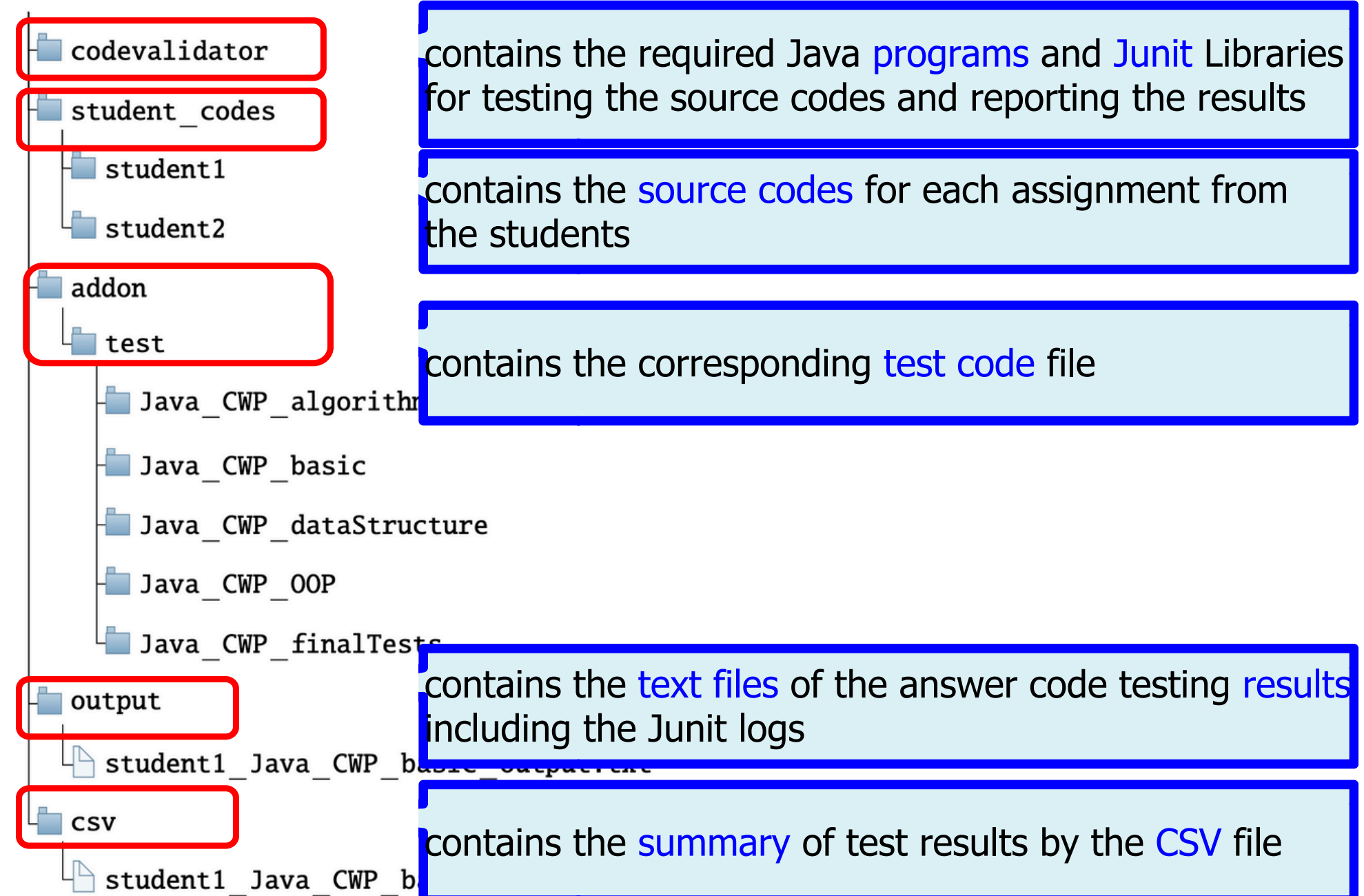
S. T. Aung, N. Funabiki, L. H. Aung, H. Htet, H. H. S. Kyaw, and S. Sugawara, "An implementation of Java programming learning assistant system platform using Node.js," in Proc. Int. Conf. Inf. Educ. Tech., pp. 47-52, 2022.

▼ Motivation and Contribution

- The teacher needs to **manually** run the **test code** and **source code** one by one to **verify** a lot of source codes from students.



answer code validation program, that can automatically verify all the source codes for one test code and report the number of tests that each code passed with the CSV file



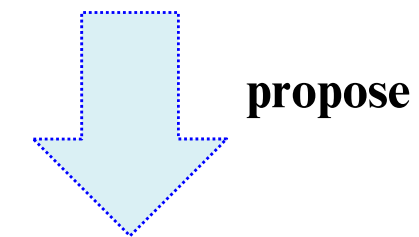
Without the program, the teacher needs to repeat to

- open the source code, run the test code on Junit, copy the result to a text file for each source code

With the program, the teacher only needs to run the program with the corresponding test code.

▼ Limitation of Test Code

- A student may use the **library** class/method or implementing the **different logic**/algorithm.
- If the student rely on the library that provides a pre-built function, the student **cannot understand** the **fundamentals** of the logic/algorithm itself.
- The current test code **cannot detect** the implementation of the different logic or algorithm if the student **use library** or **other** sorting **logic**.



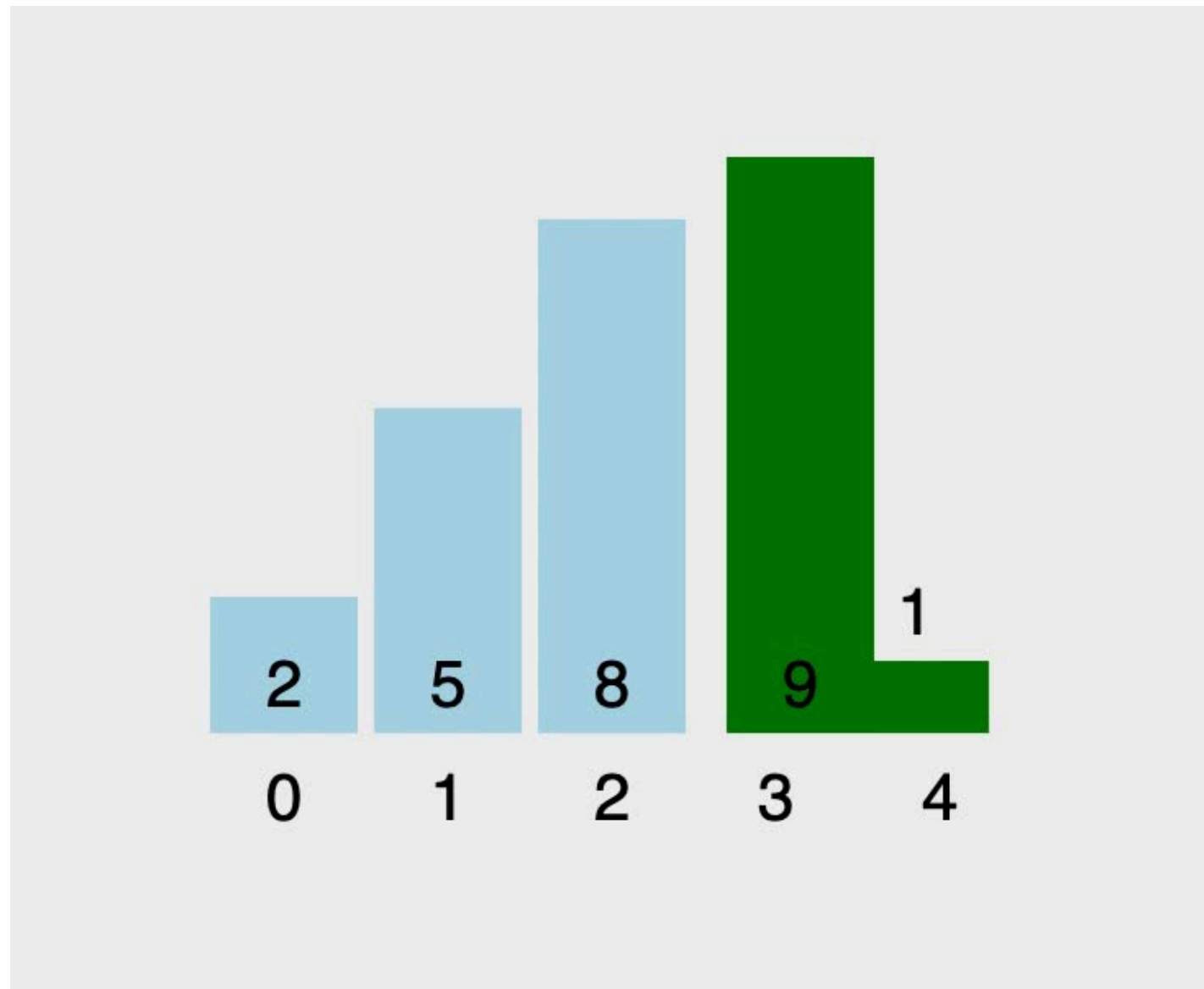
Example source code for using library

```
1: package p3;  
2: import java.util.Arrays;  
3: public class BubbleSort {  
4:     public static int[] sort(int[] a) {  
5:         Arrays.sort(a);  
6:         return a;  
7:     }  
8: }
```

intermediate state testing in the test code, that can check randomly selected intermediate state of the important variables during the execution of the logic/algorithm.

▼ Proposal of intermediate state testing

- Bubble Sort repeatedly **compares** and **swaps** adjacent elements in a list **until** the entire list is **sorted**.



Input Array: 2,5,9,8,1

Bubble Sort :

Step 1: [2, 5, 8, 1, 9]

Step 2: [2, 5, 1, 8, 9]

Step 3: [2, 1, 5, 8, 9]

Step 4: [1, 2, 5, 8, 9]

Sorted Array: 1,2,5,8,9

▼ Proposal of intermediate state testing

Example Test Code for Bubble Sort

```
package p3;
import static org.junit.Assert.*;
import org.junit.Test;
import java.util.Arrays;
public class BubbleSortTest {
    //intermediate state testing
    @Test
    public void testSortIteration() {
        BubbleSort bSort = new BubbleSort();
        int[] codeInput1 = {2,5,9,8,1};
        int codeInput2 = 2;
        int[] codeOutput = bSort.sort(
            codeInput1, codeInput2);
        int[] expOutput = {2,5,1,8,9};
        try {
            assertEquals("Test 1:", Arrays.
```

Tests an **intermediate state** of the bubble sort algorithm after a specified number of iterations.

```
//final state testing
@Test
public void testSort() {
    BubbleSort bSort = new BubbleSort();
    int[] codeInput1 = {2,5,9,8,1};
    int[] codeOutput = bSort.sort(
        codeInput1);
    int[] expOutput = {1,2,5,8,9};
    try {
        assertEquals("Test 2:", Arrays.
    } catch (Exception e) {
        fail("Test 2 failed: " + e.getMessage());
    }
}
```

Tests the **final sorted state** of the array.

Bubble Sort :

Step 1: [2, 5, 8, 1, 9]

Step 2: [2, 5, 1, 8, 9] >

Step 3: [2, 1, 5, 8, 9]

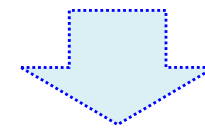
Step 4: [1, 2, 5, 8, 9] >

intermediate state #2

final state

▼ Usefulness of Intermediate State Testing in Test Code

- The intermediate state testing can
 - encourage students to **think critically** about their **source codes** and consider the **step-by-step executions** of the codes.



Provide **step-by-step** execution of the code and allow for the detection of **logical errors** or **algorithmic flaws**

▼ Limitation of Intermediate State Testing

- Source codes may **contain functions/methods** that **cannot** easily be **tested step-by-step** like intermediate state testing such as:
 - Simple Arithmetic Operations
 - Array Operations
 - String and List Operations
 - Control Structures

Test Case in the Test Code

```
5:  @Test
6:  public void testPlus(){
7:      MyMath ma = new MyMath();
8:      int codeInput1 = 1;
9:      int codeInput2 = 4;
10:     int codeOutput = ma.plus(codeInput1, codeInput2);
11:     int expOutput = 5;
12:     try {
13:         assertEquals("1:Two input case:", expOutput, codeOutput);
14:     } catch (AssertionError ae) {
15:         System.out.println(ae);
16:     }
17: }
```

Source Code (Addition)

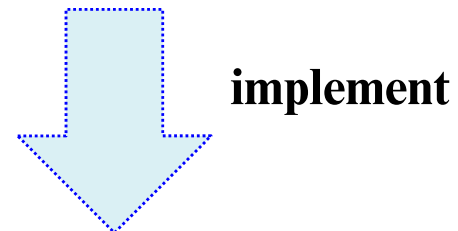
```
1: package p1;
2: public class MyMath{
3:     public int plus (int a, int b){
4:         return (a+b); 5
5:     }
6: }
```

- fixed test data in test code
- verify only the final output

test a source code with various input data

▼ Proposal of dynamic test data generation

- Unit testing **may pass** an **incorrect source code** that will **only output** the expected output described in the given **test code** without implementing the requested procedure.



test data generation algorithm, that can randomly generate and replace the input data and the expected output data for each test case in the given test code automatically.

- The dynamic test data generation algorithm is implemented in order to **dynamically change** the **test data** and replace them in the **test code**.
- The **test code** checks if the **actual output** matches the **expected output** when specific **input data** is provided.
- We need to change **input** data, **output** data, and **expected output** data in the test code.

▼ Test Code with Standard Format

- input data to the method -> **codeInput**
- output data from the method -> **codeOutput**
- expected output data -> **expOutput**

Test Code with Standard Format

```
5:  @Test
6:  public void testPlus(){
7:      MyMath ma = new MyMath();
8:      int codeInput1 = 1;
9:      int codeInput2 = 4;
10:     int codeOutput = ma.plus(codeInput1, codeInput2);
11:     int expOutput = 5;
12:     try {
13:         assertEquals("1:Two input case:", expOutput, codeOutput);
14:     } catch (AssertionError ae) {
15:         System.out.println(ae);
16:     }
17: }
```

1. Detect the **input data** from the **test code** with the standard format.
2. Find the **data type** of input data.
3. Generate the new input data randomly according to the following data type:

int	an integer number between 2 and 10
double and float	a real number between 2.0 and 10.0
int[]	array size between 5 and 10, integer number between -99 and 99
double[] and float[]	array size between 5 and 10, real number between -99 and 99
String and String[]	an English full name is randomly selected by using names library array size between 5 and 10 for String[]

*The other data type will be considered in our future works.

▼ Automatic Test Data Generation Procedure

4. Run the newly generated **test code** with the correct **source code** on Junit.

5. Find the **expected output data** from the JUnit log.

JUnit Log

.java.lang.AssertionError: 1:Two input case: expected:<5> but was:<9>

6. **Replace** the expected output data for **expOutput** in the test code by this expected output data.

```
5:  @Test
6:  public void testPlus(){
7:      MyMath ma = new MyMath();
8:      int codeInput1 = 3;
9:      int codeInput2 = 6;
10:     int codeOutput = ma.plus(codeInput1, codeInput2);
11:     int expOutput = 5;
12:     try {
13:         assertEquals("1:Two input case:", expOutput, codeOutput);
14:     } catch (AssertionError ae) {
15:         System.out.println(ae);
16:     }
17: }
```

Source Code

```
1: package p1;
2: public class MyMath{
3:     public int plus (int a, int b){
4:         return (a+b);
5:     }
6: }
```

```
5:  @Test
6:  public void testPlus(){
7:      MyMath ma = new MyMath();
8:      int codeInput1 = 3 ;
9:      int codeInput2 = 6 ;
10:     int codeOutput = ma.plus(codeInput1, codeInput2);
11:     int expOutput = 5-9;
12:     try {
13:         assertEquals("1:Two input case:", expOutput, codeOutput);
14:     } catch (AssertionError ae) {
15:         System.out.println(ae);
16:     }
17: }
```

▼ Conclusion

- String Similarity Measures for Myanmar Language
 - Phonetic Mapping
 - Sound Mapping
 - Vowel Position Mapping
 - Programming Education
 - Code Modification Problem for JavaScript Programming Education
 - Code Validation Program for Code Writing Problem in Java
- Programming Education
- Intermediate State Testing for Fundamental Algorithm Assignments
 - Implementation of Dynamic Test Data Generation Algorithm for CWP